Deep Learning
CSE641/ECE555

Lecture 15 | Take your own notes during lectures

Vinayak Abrol <abrol@iiitd.ac.in>

INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY **DELHI**

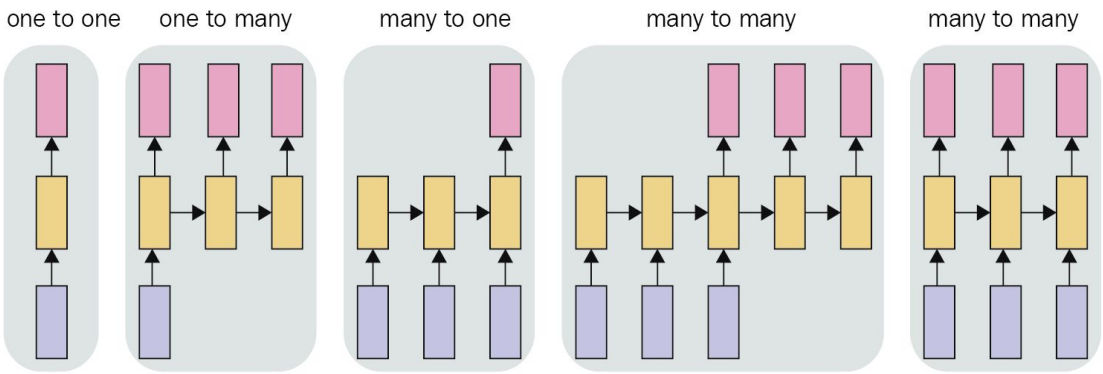
Sequence Modeling

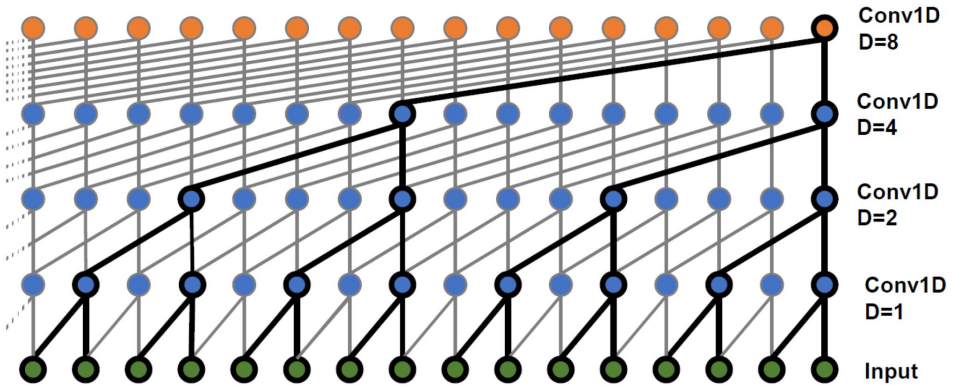
In vanilla form MLP/CNN we have seen so far are mostly of one-to-one type (e.g., image tasks).

RNN on the other hand are designed for sequences with many-to-one / many-to-many types of I/O mapping.

However, CNNs can also be used for temporal modeling!
This is because the receptive field increases with depth

CNNs allow parallel implementation but consumes more memory.

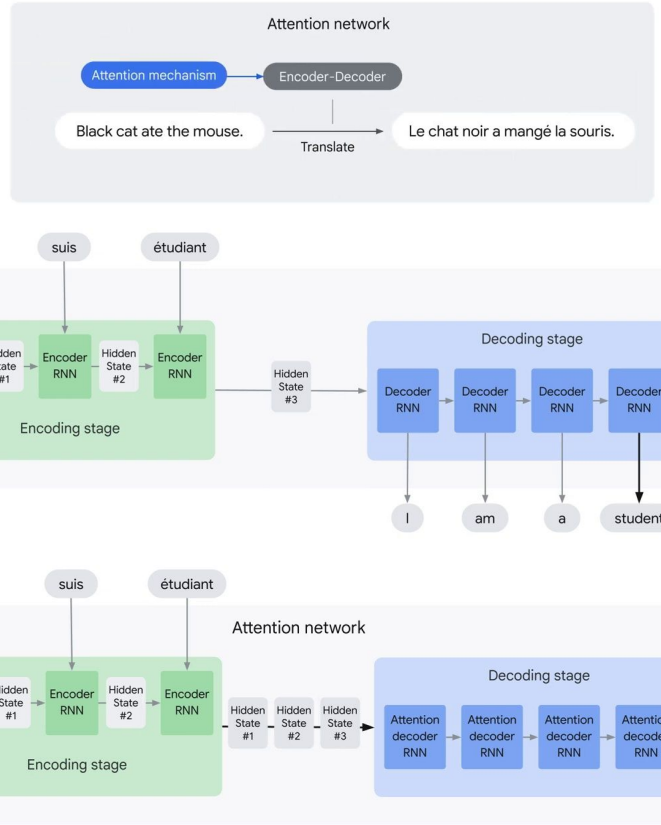




Attention Mechanism

- The core idea behind the Transformer model is the attention mechanism.
- Attention was originally proposed for encoder–decoder RNNs applied to sequence-to-sequence applications, such as machine translation.
- The intuition behind attention is that rather than one compressed representation of the input, it might be better for the decoder to revisit the input sequence at every time step.
- And selectively focus on particular parts of the input sequence at each decoding step.

https://www.youtube.com/watch?v=fjJOgb-E41w&ab_channel=GoogleCloudTech



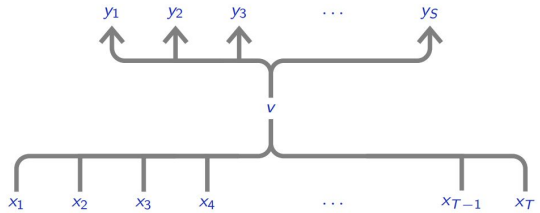
Attention Mechanism

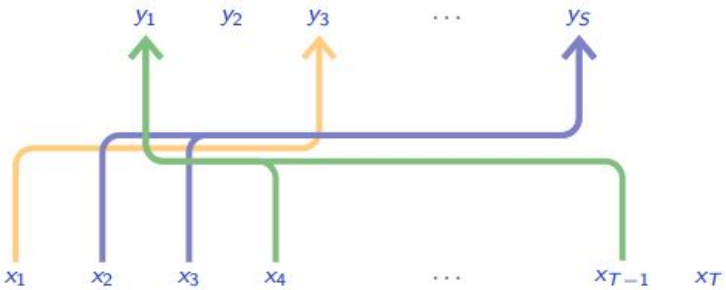
- The standard approach for sequence-to-sequence translation uses a recurrent model.
(Sutskever et al., 2014)

- The main weakness of such an approach is that all the information has to flow through a single state ‘v’, which is obtained after visiting the full input sequence.

- Attention mechanisms proposed by (Bahdanau et al., 2014) allows transport of information from parts of the input to other parts dynamically.

$$h_t = f(x_t, h_{t-1})$$
$$v = h_T$$
$$y_t \sim g(y_1 \cdots y_{t-1}, v)$$



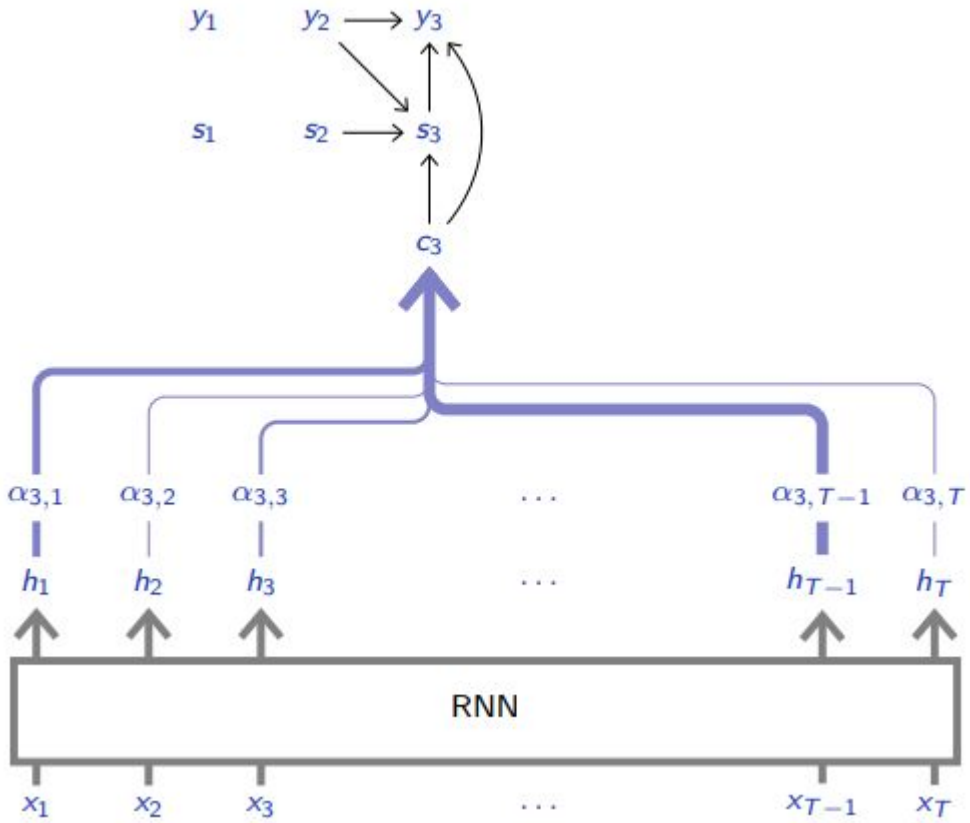


Attention Mechanism

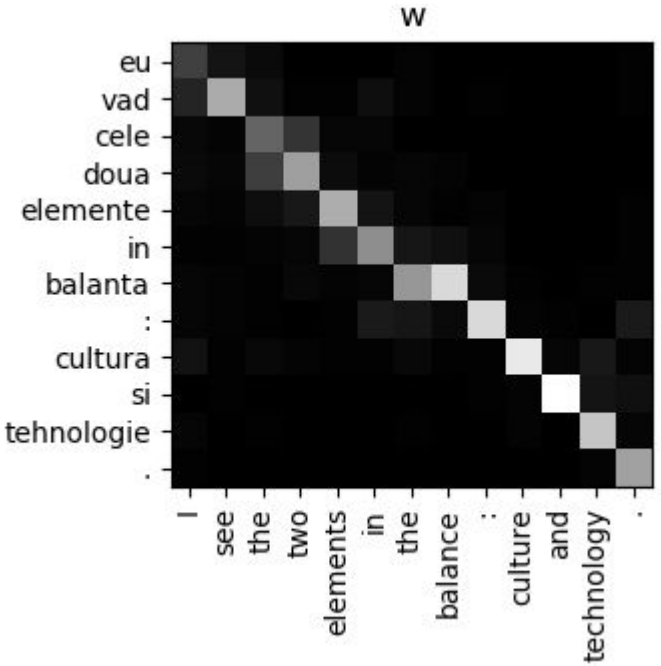
- This is context attention where s_{i-1} modulates what to look at in $h_1, \ldots, h_T$ to compute s_i and sample y_i. Here f() is a GRU.

$$h_i = \text{BLSTM}();\ i = 1 \ldots T$$
$$\forall_j, \alpha_{i,j} = \text{softmax}(\tanh(\text{FC}(h_j, s_{i-1})))$$
$$c_i = \sum_{j=1}^{T} \alpha_{i,j} h_j$$

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$
$$y_i \sim g(y_{i-1}, s_i, c_i)$$



Attention Mechanism



Intuitively, this implements a mechanism of attention in the decoder.

The decoder decides parts of the source sentence to pay attention to.

By letting the decoder have an attention mechanism, encoder is relieved from the burden of having to encode all information in the source sentence into a fixed-length vector.

(Bahdanau et al., 2014)

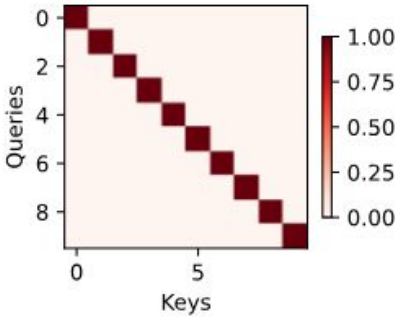
https://machinelearningmastery.com/the-bahdanau-attention-mechanism/

Queries, Keys, and Values

For long/variable sequences it becomes quite difficult to keep track of everything that has already been generated or even viewed by the network.

Consider a database: A collections of keys and values.
e.g., {(“Ram”, “Kumar”), (“Rahul”, “Bose”), (“Sumit”, “Bhat”), (“Sameera”, “Singh”), (“Ankita”, “Kumari”), (“Kalyani”, “Dutta”)}

Query Ram will retrieve value Kumar.
In general we can either have a query-key match, multiple matches (top-k), no match or approx. match.
Ideally we want query-key operation to be a valid one.



Queries, Keys, and Values

For long/variable sequences it becomes quite difficult to keep track of everything that has already been generated or even viewed by the network.

Consider a database: A collections of keys and values.
e.g., {(“Ram”, “Kumar”), (“Rahul”, “Bose”), (“Sumit”, “Bhat”), (“Sameera”, “Singh”), (“Ankita”, “Kumari”), (“Kalyani”, “Dutta”)}

Query Ram will retrieve value Kumar.
In general we can either have a query-key match, multiple matches (top-k), no match or approx. match.
Ideally we want query-key operation to be a valid one.

$$\mathcal{D} \stackrel{\text{def}}{=} \{(\mathbf{k}_1, \mathbf{v}_1), \ldots (\mathbf{k}_m, \mathbf{v}_m)\}$$

$$\text{Attention}(\mathbf{q}, \mathcal{D}) \stackrel{\text{def}}{=} \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i, \quad \sum_i \alpha(\mathbf{q}, \mathbf{k}_i) = 1 \quad \alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$$

Queries, Keys, and Values

For long/variable sequences it becomes quite difficult to keep track of everything that has already been generated or even viewed by the network.

Consider a database: A collections of keys and values.
e.g., {("Ram", "Kumar"), ("Rahul", "Bose"), ("Sumit", "Bhat"), ("Sameera", "Singh"), ("Ankita", "Kumari"), ("Kalyani", "Dutta")}

Query Ram will retrieve value Kumar.
In general we can either have a query-key match, multiple matches (top-k), no match or approx. match.
Ideally we want query-key operation to be a valid one.

$$\mathcal{D} \stackrel{\text{def}}{=} \{(\mathbf{k}_1, \mathbf{v}_1), \ldots (\mathbf{k}_m, \mathbf{v}_m)\} \qquad \alpha(\mathbf{q}, \mathbf{k}_i) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_j \exp(a(\mathbf{q}, \mathbf{k}_j))}$$

$$\text{Attention}(\mathbf{q}, \mathcal{D}) \stackrel{\text{def}}{=} \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i, \quad \sum_i \alpha(\mathbf{q}, \mathbf{k}_i) = 1 \quad \alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$$

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{1}{m} \sim \text{average pool} \qquad \alpha(\mathbf{q}, \mathbf{k}_i) = \mathbf{1}_{\mathrm{M}}(u_i) = \begin{cases} 1 & \text{when } u_i \in M \\ 0 & \text{when } u_i \notin M \end{cases} \sim \text{exact database query}$$

Queries, Keys, and Values

In general we want the attention function to be differentiable in deep learning. Exception exist in case of Reinforcement Learning but a very complex setting to optimize.

The attention function mentioned below is differentiable and its gradient never vanishes.

This idea is quite abstract and simply describes a way to pool data.

But from where these mysterious queries, keys, and values might arise from?

Regression: The query might correspond to the pixel location of regression. The keys are the nearby pixel locations we have already observed and the values are the (regression) values themselves.

$$\text{Attention}(\mathbf{q}, \mathcal{D}) \stackrel{\text{def}}{=} \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i, \quad \sum_i \alpha(\mathbf{q}, \mathbf{k}_i) = 1 \quad \alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$$

Attention Pooling by Similarity

Time to remember SVMs!

Any match can be described using similarity function or Kernel.

(Nadaraya, 1964, Watson, 1964)

Match can be performed in lower/higher dimensions.

Match can be over distributions also.

$$\alpha(\mathbf{q}, \mathbf{k}) = \exp\left(-\frac{1}{2\sigma^2}\|\mathbf{q} - \mathbf{k}\|^2\right) \quad \text{Gaussian;}$$

$$\alpha(\mathbf{q}, \mathbf{k}) = 1 \text{ if } \|\mathbf{q} - \mathbf{k}\| \leq 1 \quad \text{Boxcar;}$$

$$\alpha(\mathbf{q}, \mathbf{k}) = \max(0, 1 - \|\mathbf{q} - \mathbf{k}\|) \quad \text{Epanechikov.}$$

$$f(\mathbf{q}) = \sum_i \mathbf{v}_i \frac{\alpha(\mathbf{q}, \mathbf{k}_i)}{\sum_j \alpha(\mathbf{q}, \mathbf{k}_j)}.$$

- Example: Observations $(\mathbf{x}_i, \mathbf{y}_i)$ for features and labels, $\mathbf{v}_i = \mathbf{y}_i$ are scalars/one-hot vectors, $\mathbf{k}_i = \mathbf{x}_i$ are vectors, and the query $\mathbf{q}$ denotes the new location where **f()** should be evaluated.

- Advantages of this estimator is that it requires no training.
- A much better strategy is to learn the mechanism, by learning the representations for queries and keys.

Homework:

Prove that all kernels above are translation and rotation invariant; that is, if we shift $\mathbf{q}$ and $\mathbf{k}$ rotate and in the same manner, the value of $\boldsymbol{\alpha}$ remains unchanged.

Attention Scoring Functions

Distance functions are slightly more expensive to compute than dot products.

$$\alpha(\mathbf{q}, \mathbf{k}) = \exp\left(-\frac{1}{2\sigma^2}\|\mathbf{q} - \mathbf{k}\|^2\right) \quad \text{Gaussian;}$$

$$\alpha(\mathbf{q}, \mathbf{k}_i) = -\frac{1}{2}\|\mathbf{q} - \mathbf{k}_i\|^2 = \mathbf{q}^\top \mathbf{k}_i - \frac{1}{2}\|\mathbf{k}_i\|^2 - \frac{1}{2}\|\mathbf{q}\|^2.$$

Consider attention weights without exponential

- Normalizing **α to 1** will make 3rd term disappear. Because the final term depends on **q** only & thus it is identical for all **(q,k_i)** pairs.
- BN/LN leads to well bounded outputs, so 2nd term is approximately constant and can be dropped.
- The remaining first term is just the dot product.

$$a(\mathbf{q}, \mathbf{k}_i) = \mathbf{q}^\top \mathbf{k}_i / \sqrt{d}.$$

$$\mathbf{q} \in \mathbb{R}^d \quad \mathbf{k}_i \in \mathbb{R}^d$$

what about mean?

- In practice we use scaled dot product to ensure variance of the dot product still remains 1, assuming Keys & Queries are IID.

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \mathrm{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(\mathbf{q}^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=1} \exp(\mathbf{q}^\top \mathbf{k}_j / \sqrt{d})}.$$

This is the idea in
the famous Transformer paper (Vaswani et al., 2017)

Practical Considerations

Masked Softmax Operation

Limit the softmax to actual length of sequence.

An A+ in Deep Learning
Learn to code <blank> <blank>
Maths coming <blank> <blank> <blank>

```
masked_softmax(torch.rand(2,2,4),torch.tensor([[
1,3], [2,4]]))

tensor([[[1.0000, 0.0000, 0.0000, 0.0000],
         [0.4109, 0.2794, 0.3097, 0.0000]],

        [[0.3960, 0.6040, 0.0000, 0.0000],
         [0.2557, 0.1833, 0.2420, 0.3190]]])
```

```
def masked_softmax(X, valid_lens):
    # X: 3D tensor, valid_lens: 1D or 2D tensor
  def_sequence_mask(X, valid_len, value=0):
      maxlen = X.size(1)
      mask=torch.arange((maxlen))[None,:]<valid_len[:,None]
      X[~mask] = value
    return X

  if valid_lens is None:
    return nn.functional.softmax(X, dim=-1)
  else:
    shape = X.shape
    if valid_lens.dim() == 1:
      valid_lens=torch.repeat_interleave(valid_lens, shape[1])
    else:
      valid_lens = valid_lens.reshape(-1)

# On the last axis, replace masked elements with a very large
negative value, whose exponentiation outputs 0
     X   =   _sequence_mask   (X.reshape(-1,shape[-1]),valid_lens,
value=-1e6)
 return nn.functional.softmax(X.reshape(shape), dim=-1)
```

Practical Considerations

Batch Matrix Multiplication

$$\mathbf{Q} = [\mathbf{Q}_1, \mathbf{Q}_2, \ldots, \mathbf{Q}_n] \in \mathbb{R}^{n \times a \times b},$$
$$\mathbf{K} = [\mathbf{K}_1, \mathbf{K}_2, \ldots, \mathbf{K}_n] \in \mathbb{R}^{n \times b \times c}.$$

$$\mathrm{BMM}(\mathbf{Q}, \mathbf{K}) = [\mathbf{Q}_1\mathbf{K}_1, \mathbf{Q}_2\mathbf{K}_2, \ldots, \mathbf{Q}_n\mathbf{K}_n] \in \mathbb{R}^{n \times a \times c}.$$

Scaled Dot Product Attention

$\mathbf{q}^\top \mathbf{k}$ $\mathbf{q}^\top \mathbf{M} \mathbf{k}$

We use a transform matrix ‘M’ when length of key & query don’t match.

$$\mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \in \mathbb{R}^{n \times v}.$$

$\mathbf{Q} \in \mathbb{R}^{n \times d}$ $\mathbf{K} \in \mathbb{R}^{m \times d}$ $\mathbf{V} \in \mathbb{R}^{m \times v}$

```
class DotProductAttention(nn.Module):
    """Scaled dot product attention."""
    def __init__(self, dropout):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

# Shape of queries: (batch_size, no. of queries, d)
# Shape of keys: (batch_size, no. of key-value pairs, d)
# Shape of values: (batch_size, no. of key-value pairs, value dimension)
# Shape of valid_lens: (batch_size,) or (batch_size, no. of queries)
    def forward(self, queries, keys, values,valid_lens=None):
        d = queries.shape[-1]
# Swap the last two dimensions of keys with keys.transpose(1, 2)
        scores = torch.bmm(queries, keys.transpose(1, 2))/
        math.sqrt(d)
        self.attention_weights = masked_softmax(scores,
        valid_lens)
        return torch.bmm(self.dropout(self.attention_weights),
                values)
```

Practical Considerations

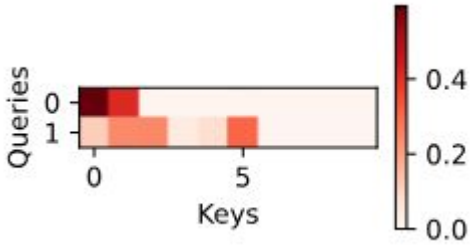
Additive Attention

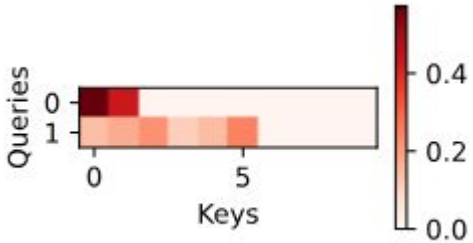
$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k}) \in \mathbb{R},$$

$$\mathbf{W}_q \in \mathbb{R}^{h \times q}, \quad \mathbf{W}_k \in \mathbb{R}^{h \times h}, \quad \mathbf{w}_v \in \mathbb{R}^h$$

Does this looks similar?

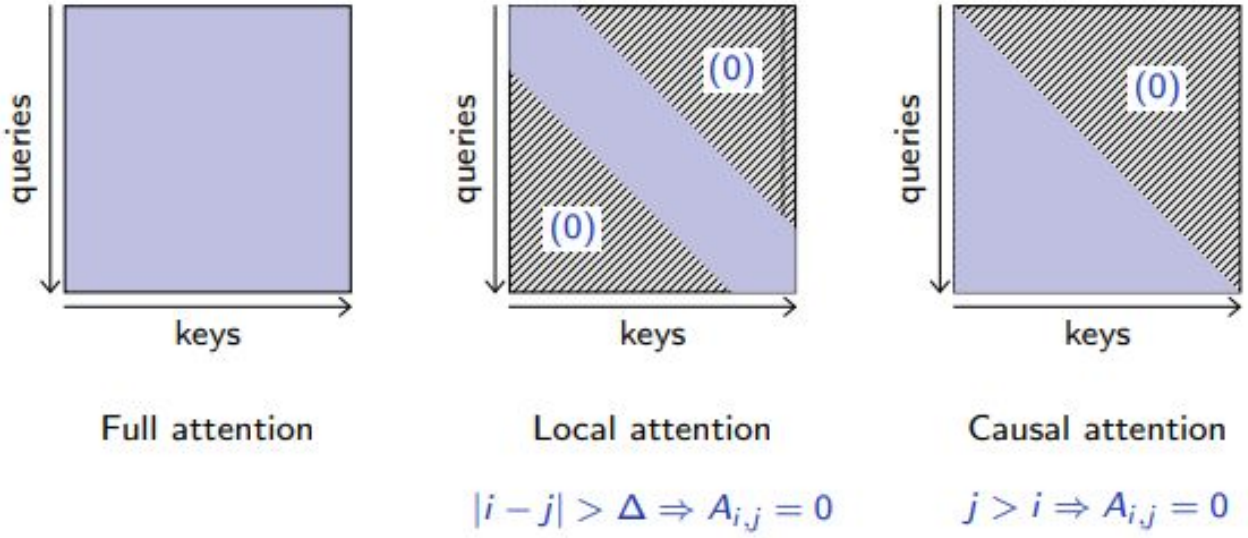
```
#we have a minibatch size of 2, a total of 10
keys and values with dimensionality of 2 & 4.
Query is 2 dimensional.

queries = torch.normal(0, 1, (2, 1, 2))
keys = torch.normal(0, 1, (2, 10, 2))
values = torch.normal(0, 1, (2, 10, 4))
valid_lens = torch.tensor([2, 6])

attention.attention_weights.reshape((1,1,2,10))
```





Bahdanau attention is often referred to as "additive attention" because it uses a feedforward network to compute alignment scores, which is a form of additive operation. Not Additive Attention as above.

Practical Considerations



In the case of local attention (Beltagy et al., 2020), only keys stored near the query in the sequence are considered.

For causal attention (Vaswani et al., 2017), only keys stored before the query in the sequence are considered.

Practical Considerations

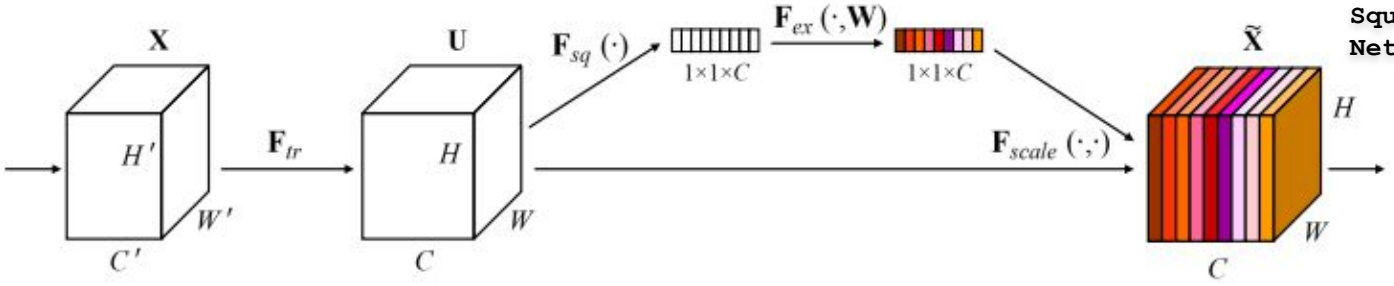
The computational cost of self-attention is:

O(n d^2) for computing the query, key, and value matrices by linearly transforming the input matrix.
O(n^2 d) for computing the dot product between the query and key matrices.
O(n^2 d) for computing the weighted sum of the value matrix.
Therefore, the total computational cost of self-attention is O(n^2 d + n d^2), which scales quadratically with n and d. (This is practical complexity and different than what is mentioned in paper. In wall clock time Transformer was found to be faster to train.)

The memory bandwidth requirements of self-attention are:

O(n d) for storing the input matrix.
O(n d) for storing the query, key, and value matrices.
O(n^2) for storing the attention matrix.
O(n d) for storing the output matrix.
Therefore, the total memory bandwidth requirements of self-attention are O(n^2 + n d), which scales quadratically with n and linearly with d.

Dynamic CNNs via Attention



Squeeze and Excitation Networks (Hu et al., 2018)

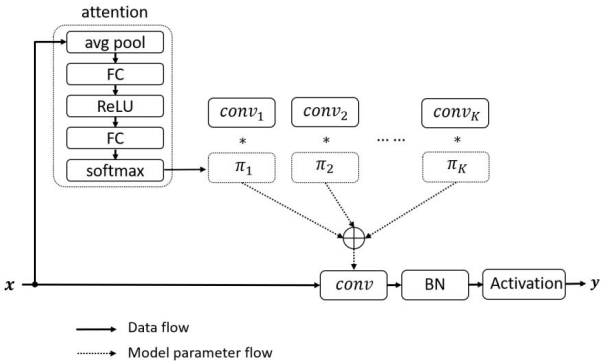
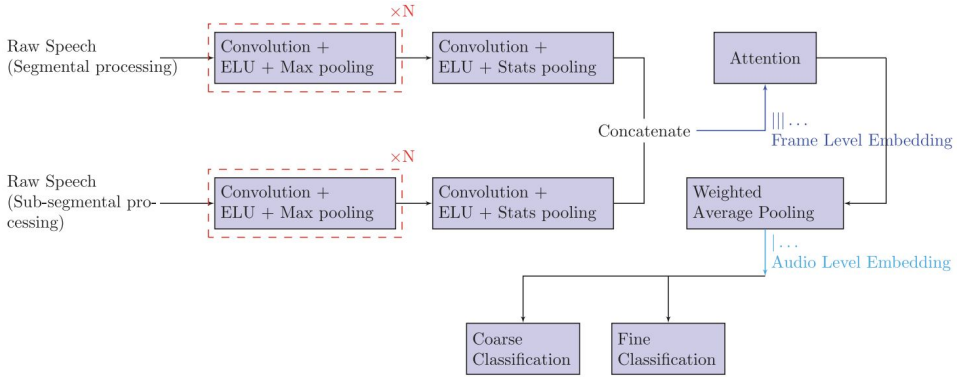


Figure 3. A dynamic convolution layer.

Dynamic Convolution: Attention over Convolution Kernels (Chen et al., 2019)



(Abrol et al., 2020)

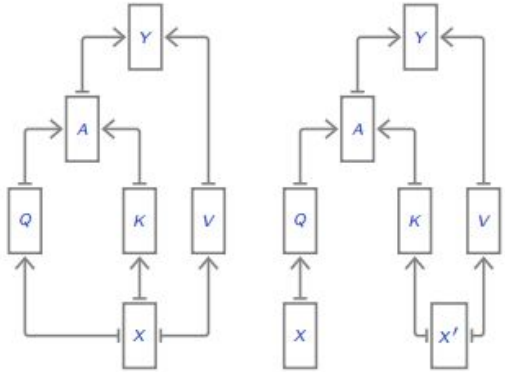
Attention Layers

A standard attention layer takes as input two sequences X and X′ and computes the tensors K, V, and Q as per-row linear functions.

When X = X′, this is self attention, otherwise it is cross attention.

Multi-head Attention combines several such operations in parallel, and Y is the concatenation of the results along the feature dimension to which is applied one more linear transformation.

$$Q = XW^{Q\top}$$
$$K = X'W^{K\top}$$
$$V = X'W^{V\top}$$
$$A = \text{softmax}_{row}\left(\frac{QK^\top}{\sqrt{D}}\right)$$
$$Y = AV$$



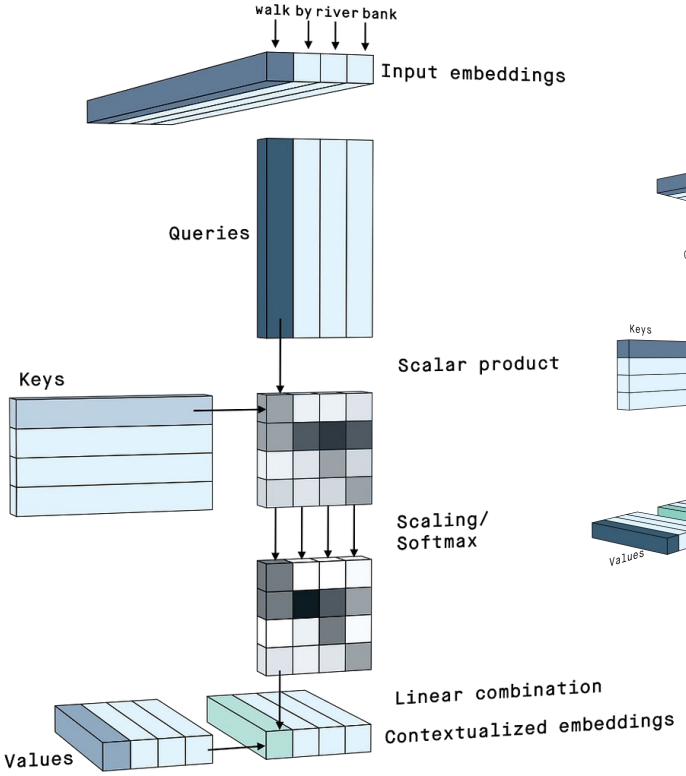
The standard attention operation is invariant to a permutation of the keys and values:

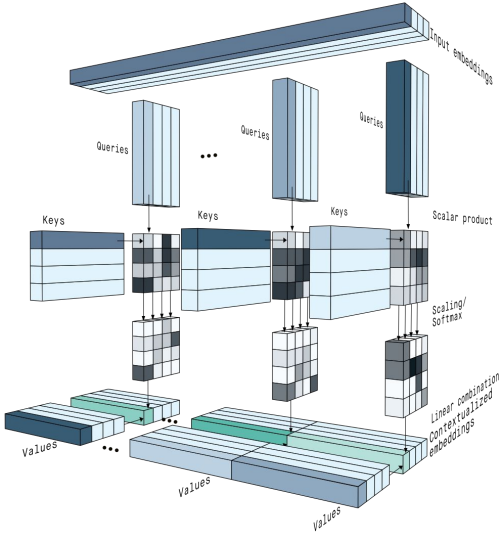
$$Y(Q, \sigma(K), \sigma(V)) = Y(Q, K, V),$$

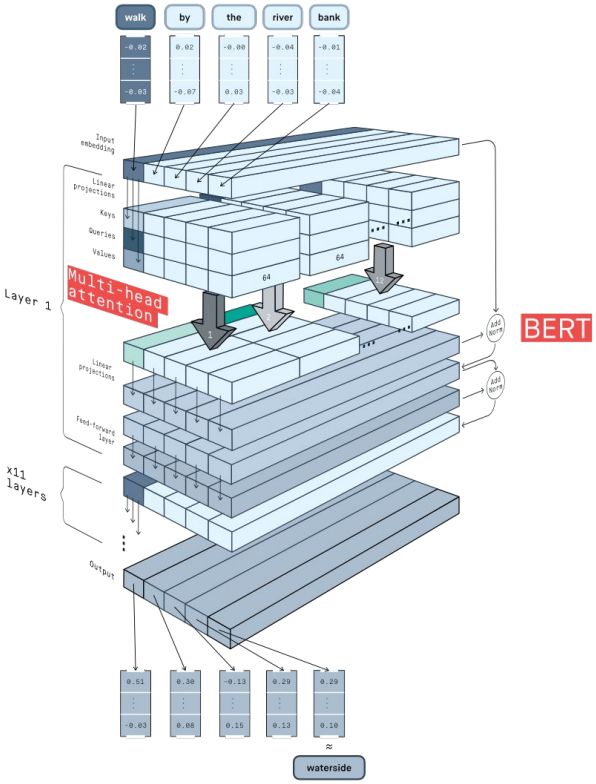
and equivariant to a permutation of the queries, that is the resulting tensor is permuted similarly:

$$Y(\sigma(Q), K, V) = \sigma(Y(Q, K, V)).$$

Snapshot







https://towardsai.net/p/nlp/getting-meaning-from-text-self-attention-step-by-step-video

Reading

D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. CoRR, abs/1409.0473, 2014.

K. Cho, B. van Merrienboer, C¸. G¨ul¸cehre, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. CoRR, abs/1406.1078, 2014.

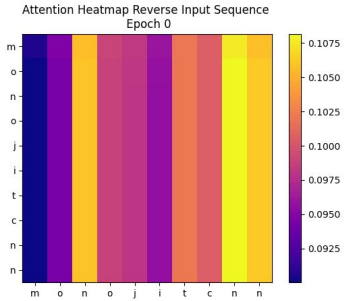
I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In Neural Information Processing Systems (NIPS), pages 3104–3112, 2014.

I. Beltagy, M. Peters, and A. Cohan. Longformer: The long-document transformer. CoRR, abs/2004.05150, 2020.

A. Graves, G. Wayne, and I. Danihelka. Neural turing machines. CoRR, abs/1410.5401, 2014.

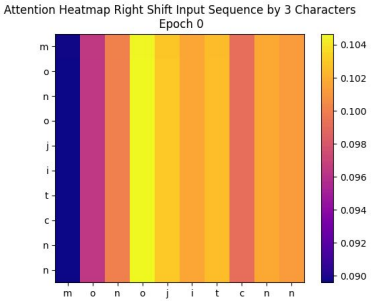
A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. CoRR, abs/1706.03762, 2017.

Attention Maps

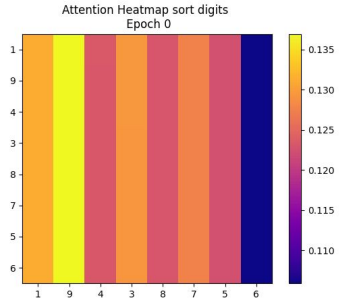


Reverse a sequence

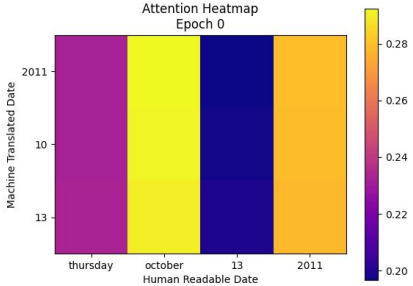
Left shift a sequence by 4 characters



Right shift a sequence by 3 characters

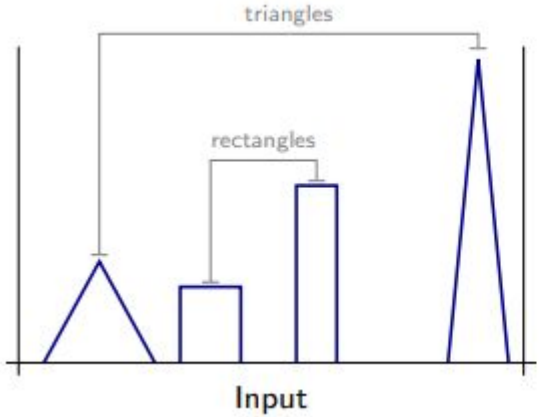


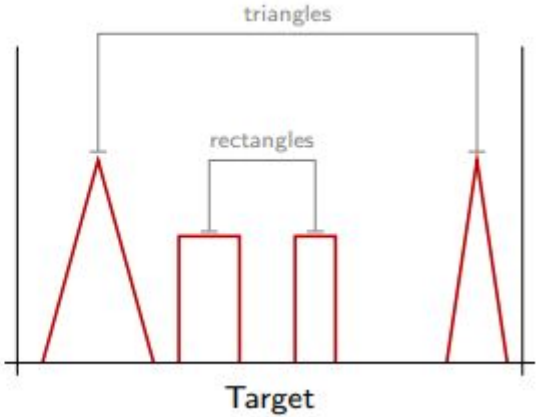
Sort digits in ascending order



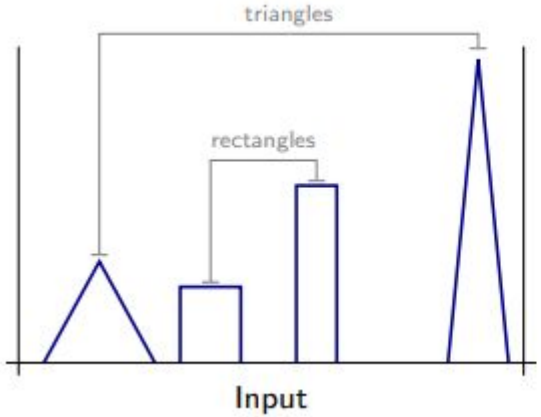
Date format conversion

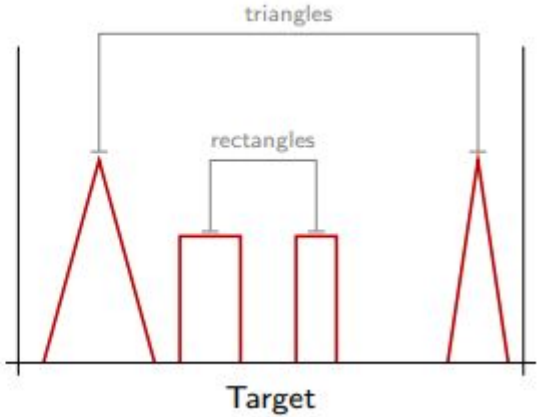
Example Problem

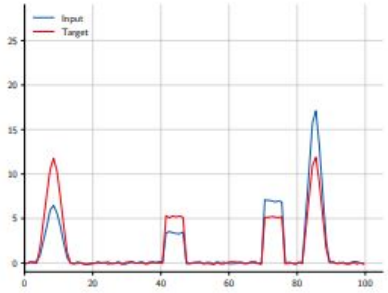


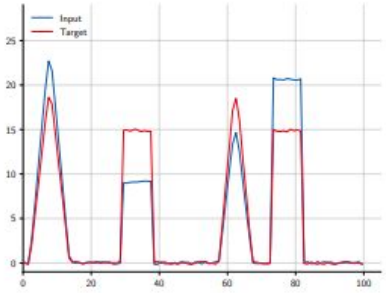


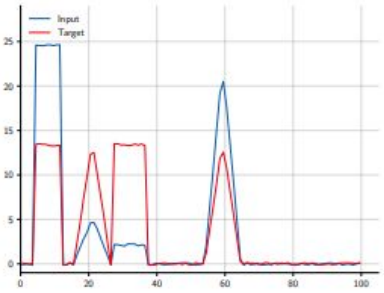
Example Problem

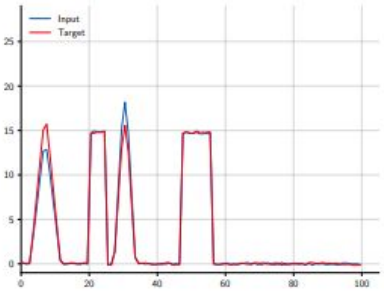






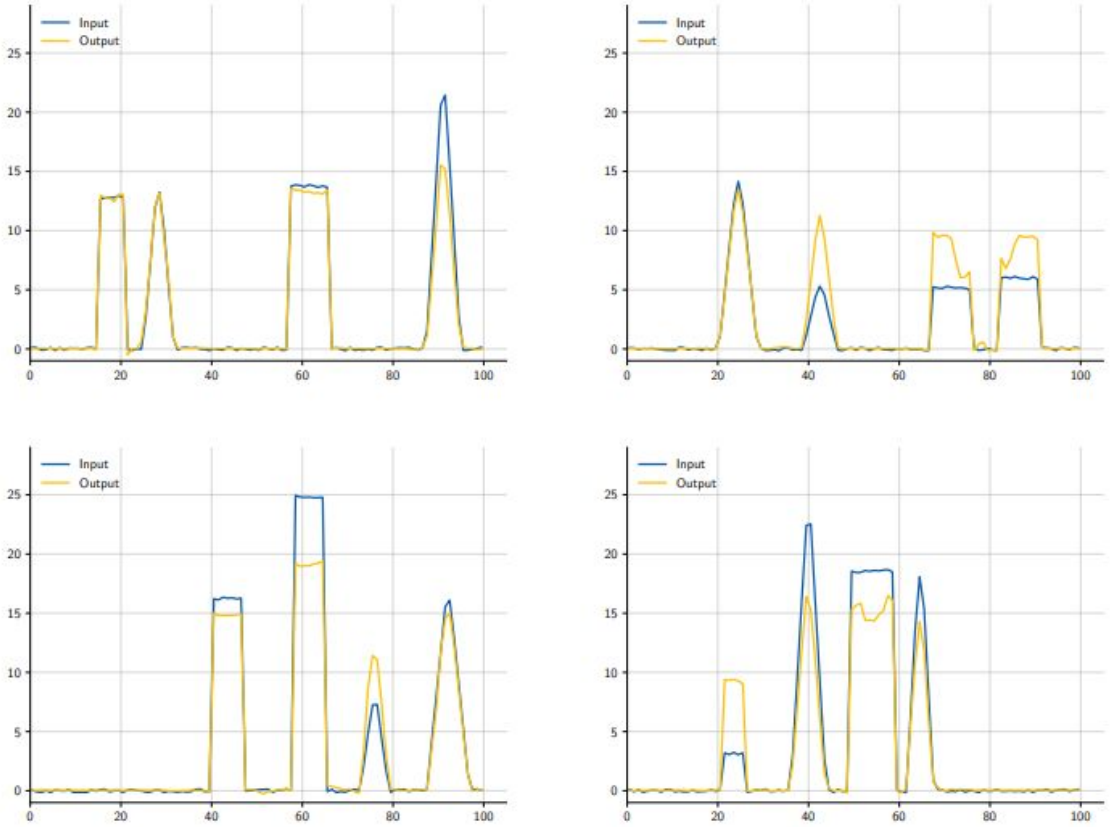






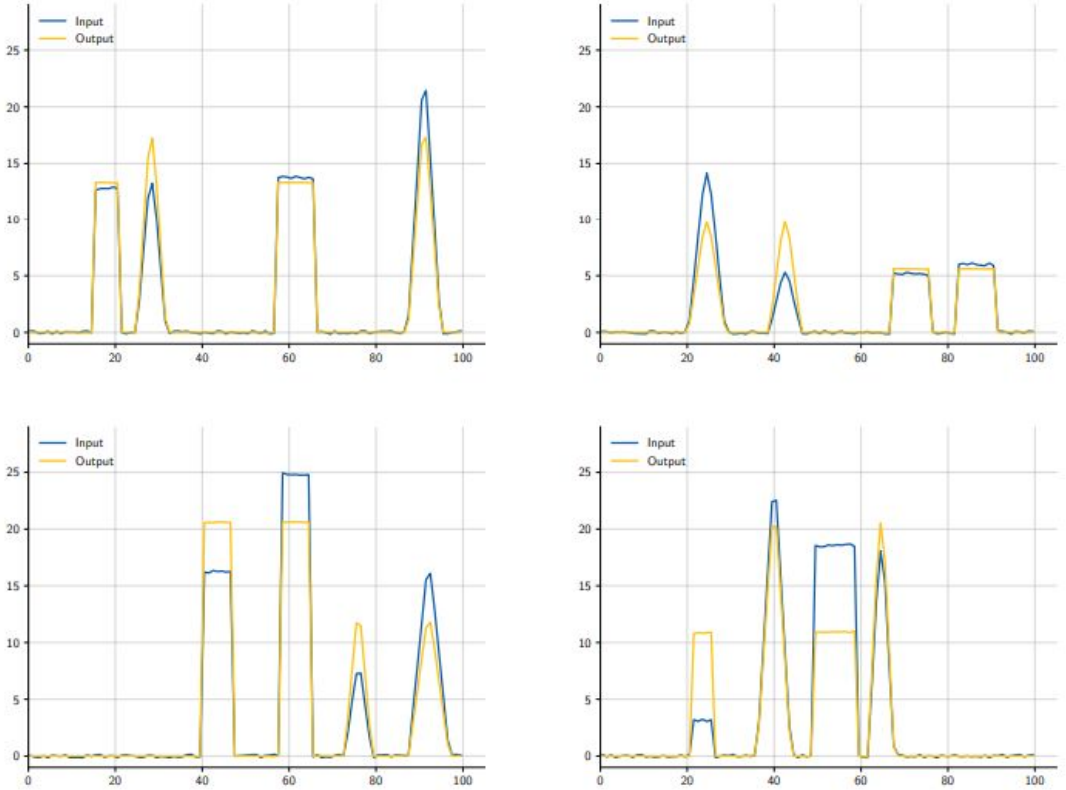
Example Problem

4 layer CNN without attention.

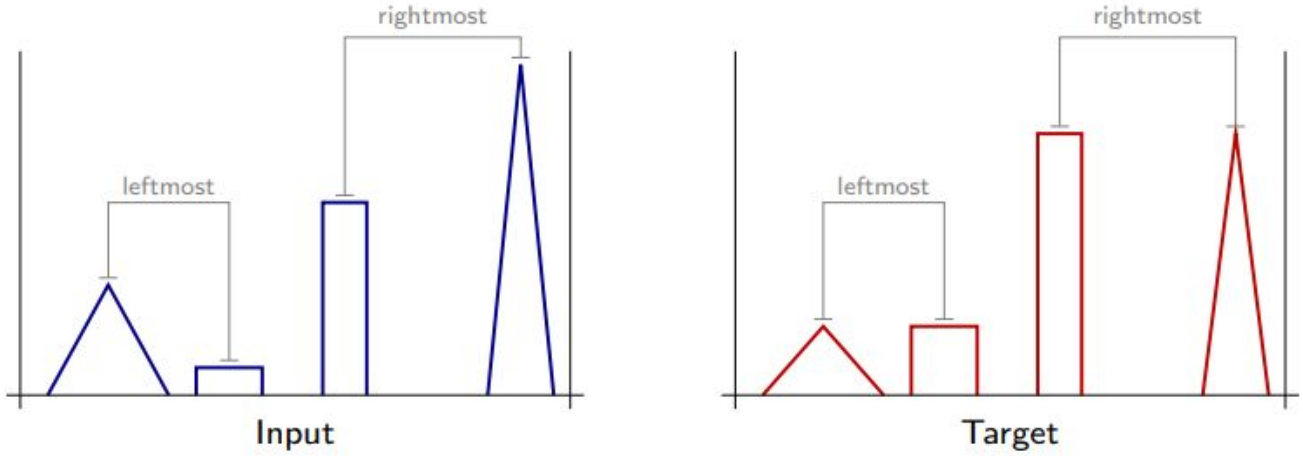


Example Problem

4 layer CNN with attention.



Example Problem 2

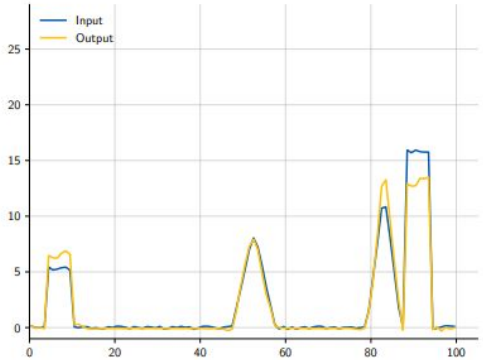


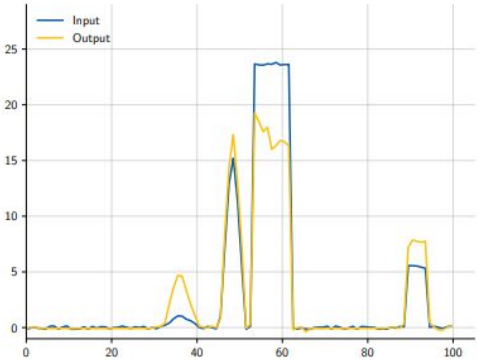
Remember the invariance to a permutation of the keys and values.

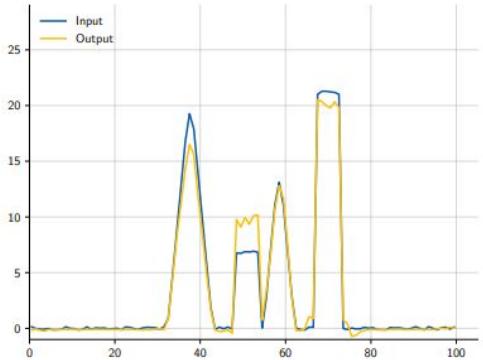
Example Problem 2

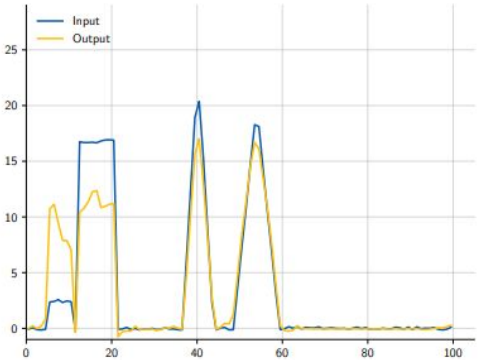
Attention doesn't helps here!

Why?



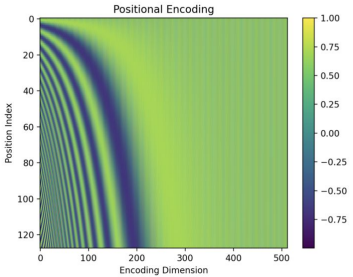


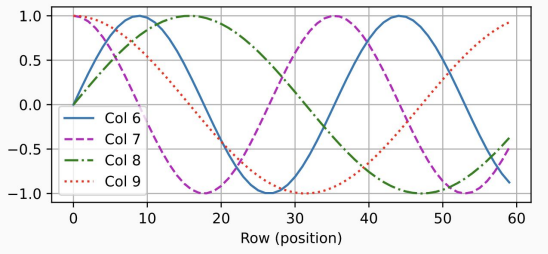




Example Problem 2

Attention doesn't helps here!
We can fix this by providing to the model a **positional encoding.**





Position is encoded as one-hot vector and concatenated as channels.

(0, 0, 0, 0, 0), (0, 0, 0, 0, 1) .....

There are other ways also such as sinusoidal or relative encoding. It can be learned also. And in practice we use additive encoding rather than concatenative.

-It should output a unique encoding for each time-step.
-Distance between any two time-steps should be consistent across sentences with different lengths.
-Our model should generalize to longer sentences without any efforts.
-Its values should be bounded.
-It must be deterministic.

Residual connection ensures the position information propagates deeper in network.

Example Problem 2

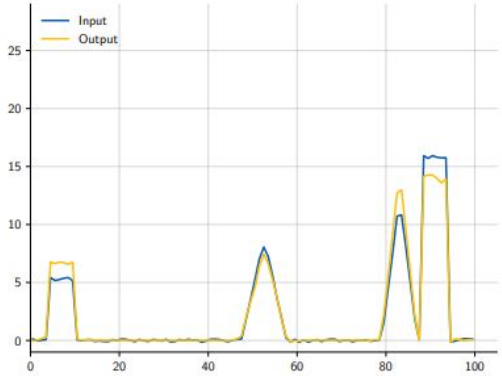
Attention doesn't helps here!

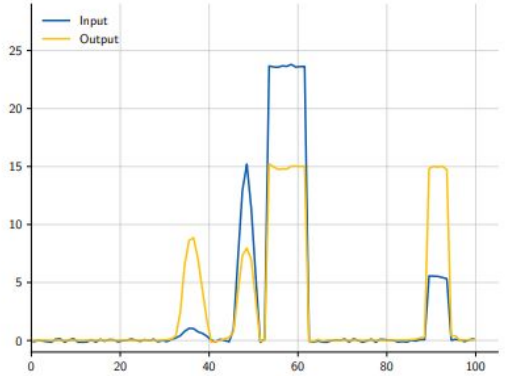
We can fix this by providing to the model a **positional encoding.**

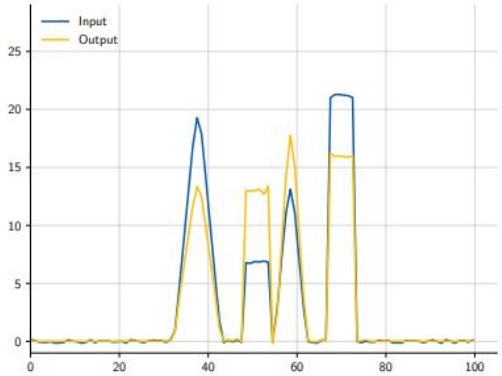
Position is encoded as one-hot vector and concatenated as channels.

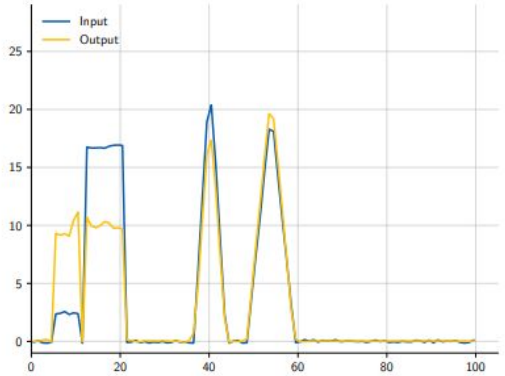
(0, 0, 0, 0, 0), (0, 0, 0, 0, 1) .....

There are other ways also such as sinusoidal or relative encoding.









Transformer Architecture

Unlike earlier attention models that rely on RNNs for input representations, the Transformer model is solely based on attention mechanisms without any convolutional or recurrent layer.

The encoder outputs a vector representation for each position of the input sequence.

The decoder uses an additional intermediate cross-attention layer. Also it uses masked/casual attention.

In seq-tasks the sentence length often varies & thus, for batchnorm, it would be uncertain what would be the appropriate normalization constant. Feature dimension is fixed and hence LayerNorm. Also there is additional need for synchronization for BN to be truly parallel.

More in NLP or ADL course